REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 4
Date 20.03.26
Topic: "Sampling and Reconstruction of Signals: Analysis of Aliasing Effects and Proper Signal Reconstruction. 8. Coding and Decoding Digital Signals"
Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13.

Solve Problem 4: Compare signal distortion and compression ratio for thresholds of 1, 5, and 10 in DCT compression for the signal [2, 4, 6, 8, 10, 12].

2. Input data:

signal = np.array([2, 4, 6, 8, 10, 12])
thresholds = [1, 5, 10]

3. Commands used (or GUI):

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import dct, idct

# --- Input signal ---
signal = np.array([2, 4, 6, 8, 10, 12])


# --- DCT compression function ---
def dct_compress(signal, threshold):
    # Apply Discrete Cosine Transform (DCT)
    coeffs = dct(signal, norm='ortho')

    # Thresholding: set small coefficients to zero
    compressed_coeffs = np.where(np.abs(coeffs) > threshold, coeffs, 0)

    # Reconstruct signal using inverse DCT
    reconstructed = idct(compressed_coeffs, norm='ortho')

    return coeffs, compressed_coeffs, reconstructed


# --- Function to analyze compression vs distortion ---
def analyze_tradeoff(signal, thresholds):
    results = {
        "thresholds": [],
        "compression_ratios": [],
        "distortions": []
    }

    original_size = len(signal)

    for threshold in thresholds:
        coeffs, compressed_coeffs, reconstructed = dct_compress(signal,
threshold)

        compressed_size = np.count_nonzero(compressed_coeffs)
        compression_ratio = original_size / compressed_size if compressed_size
!= 0 else np.inf
        mse = np.mean((signal - reconstructed) ** 2)

        results["thresholds"].append(threshold)
        results["compression_ratios"].append(compression_ratio)
        results["distortions"].append(mse)
```

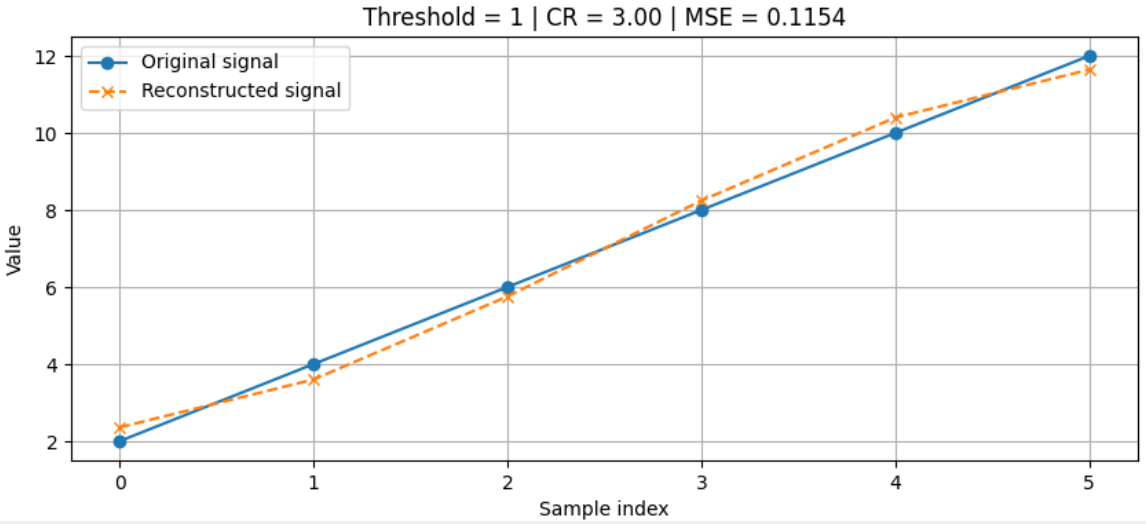
```
        # --- PLOT FOR EACH THRESHOLD ---
        plt.figure(figsize=(10, 4))

        # Original vs reconstructed signal
        plt.plot(signal, marker='o', label='Original signal')
        plt.plot(reconstructed, marker='x', linestyle='--',
label='Reconstructed signal')

        plt.title(f"Threshold = {threshold} | CR = {compression_ratio:.2f} |
MSE = {mse:.4f}")
        plt.xlabel("Sample index")
        plt.ylabel("Value")
        plt.legend()
        plt.grid()
        plt.show()

        # --- DCT coefficients plot ---
        plt.figure(figsize=(10, 4))

        plt.stem(coeffs, basefmt=" ", label='Original DCT')
        plt.stem(compressed_coeffs, linefmt='r-', markerfmt='ro', basefmt=" ",
label='After threshold')

        plt.title(f"DCT Coefficients (Threshold = {threshold})")
        plt.xlabel("Coefficient index")
        plt.ylabel("Value")
        plt.legend()
        plt.grid()
        plt.show()

    return results


# --- Threshold values ---
thresholds = [1, 5, 10]

# --- Run analysis ---
results = analyze_tradeoff(signal, thresholds)

# --- Print results ---
print("Thresholds:", results["thresholds"])
print("Compression Ratios:", results["compression_ratios"])
print("Distortions (MSE):", results["distortions"])
```

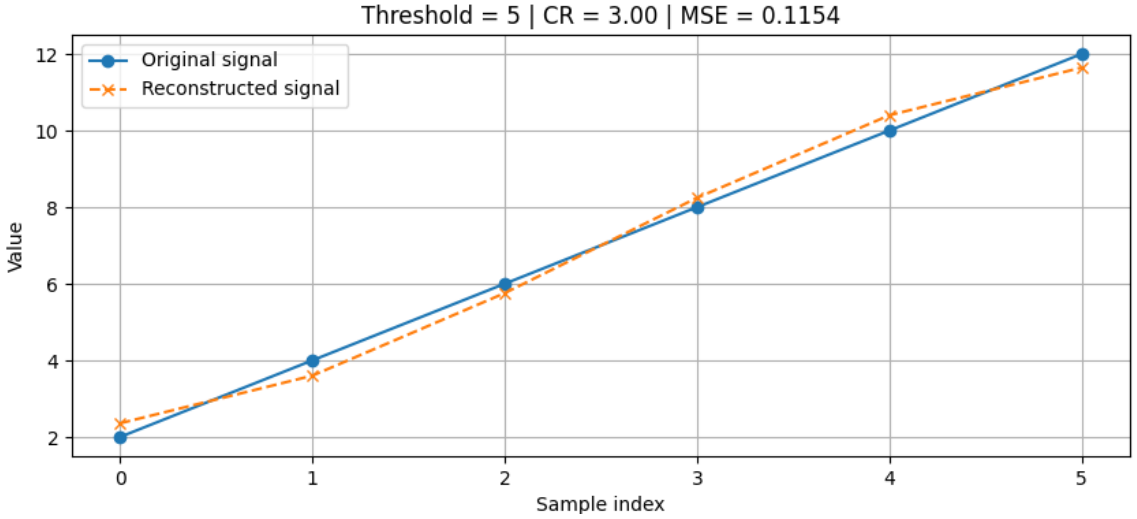
Output:

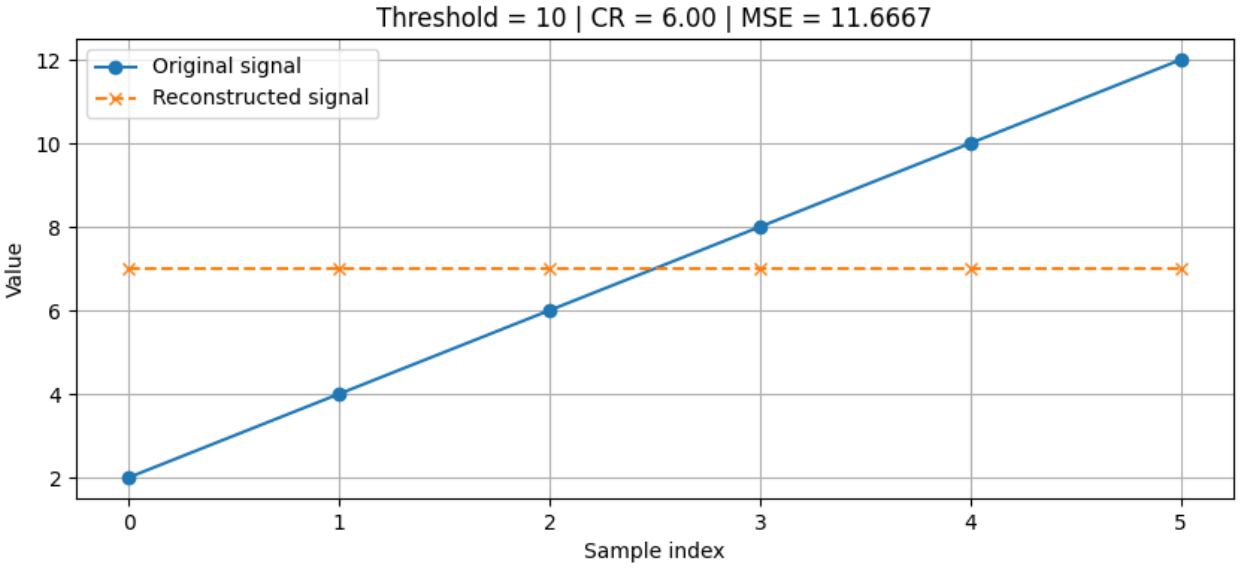
Thresholds: [1, 5, 10]

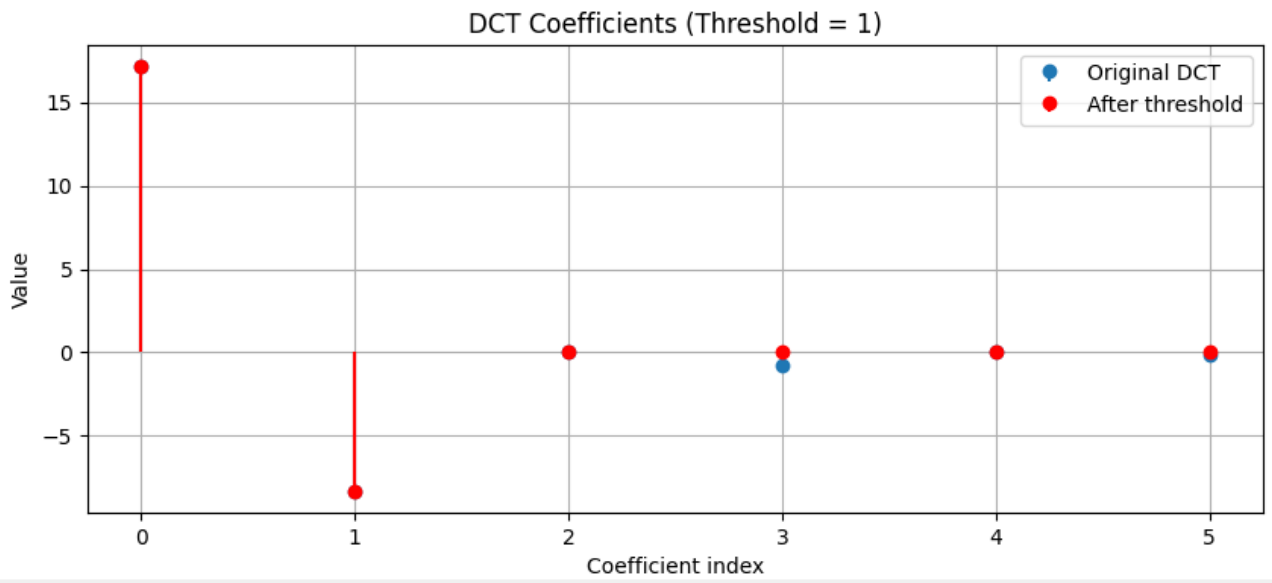
Compression Ratios: [np.float64(3.0), np.float64(3.0), np.float64(6.0)]

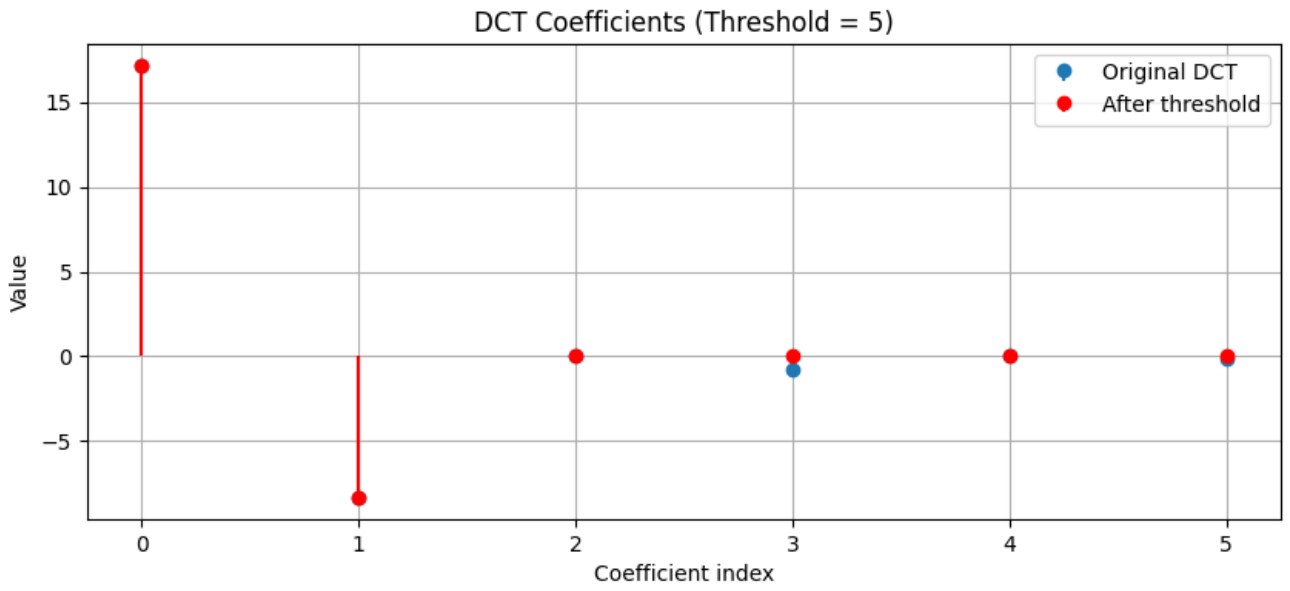
Distortions (MSE): [np.float64(0.11538619699263104), np.float64(0.11538619699263104), np.float64(11.666666666666666)]

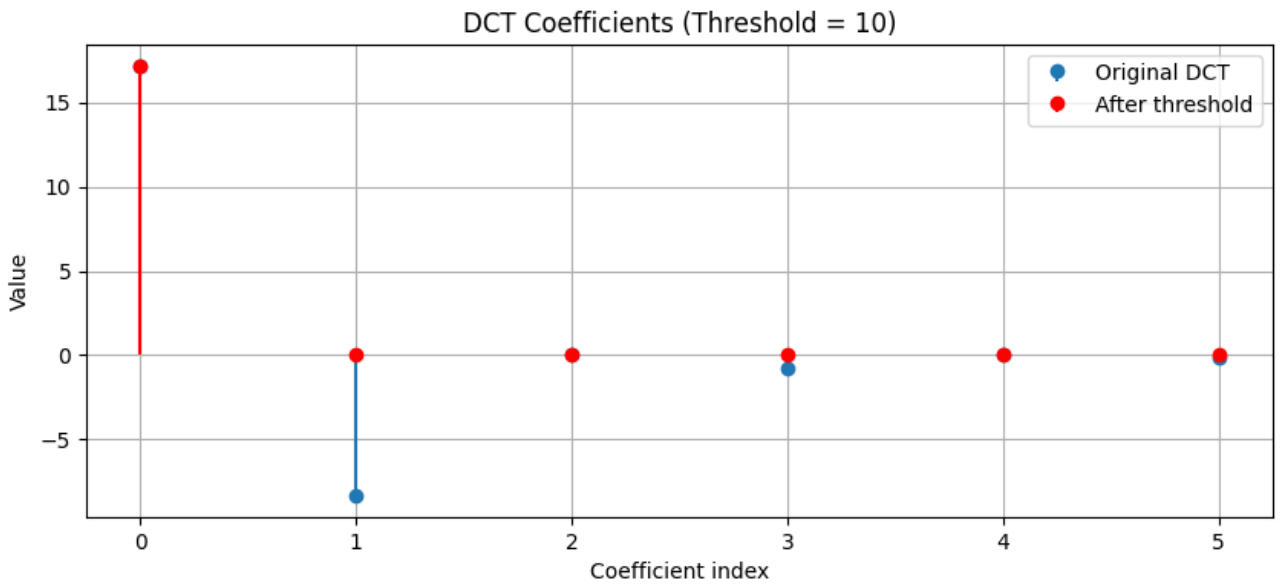

Threshold = 1 | CR = 3.00 | MSE = 0.1154
Original signal
Reconstructed signal
Value
Sample index


Threshold = 5 | CR = 3.00 | MSE = 0.1154
Original signal
Reconstructed signal
Value
Sample index


Threshold = 10 | CR = 6.00 | MSE = 11.6667
Original signal
Reconstructed signal
Value
Sample index


DCT Coefficients (Threshold = 1)
Original DCT
After threshold
Value
Coefficient index
15
10
5
0
−5
0
1
2
3
4
5


DCT Coefficients (Threshold = 5)
Original DCT
After threshold
Value
Coefficient index
15
10
5
0
−5
0
1
2
3
4
5


DCT Coefficients (Threshold = 10)
Original DCT
After threshold
Value
Coefficient index
15
10
5
0
−5
0
1
2
3
4
5

The results show that increasing the threshold improves the compression ratio, but at the cost of significantly higher distortion. While thresholds 1 and 5 maintain low error with moderate compression, threshold 10 achieves higher compression but introduces a substantial loss in signal quality.

Conclusion:

Due to this lab, we learned how DCT-based compression affects signal quality and data reduction. We observed that applying a low threshold preserves most of the signal information with minimal distortion, while higher thresholds remove more coefficients and increase compression efficiency. We also saw that excessive thresholding leads to significant reconstruction error, highlighting the trade-off between compression ratio and signal fidelity in practical applications.

Link to remote repozytorium (e.g. GitHub):

https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%204